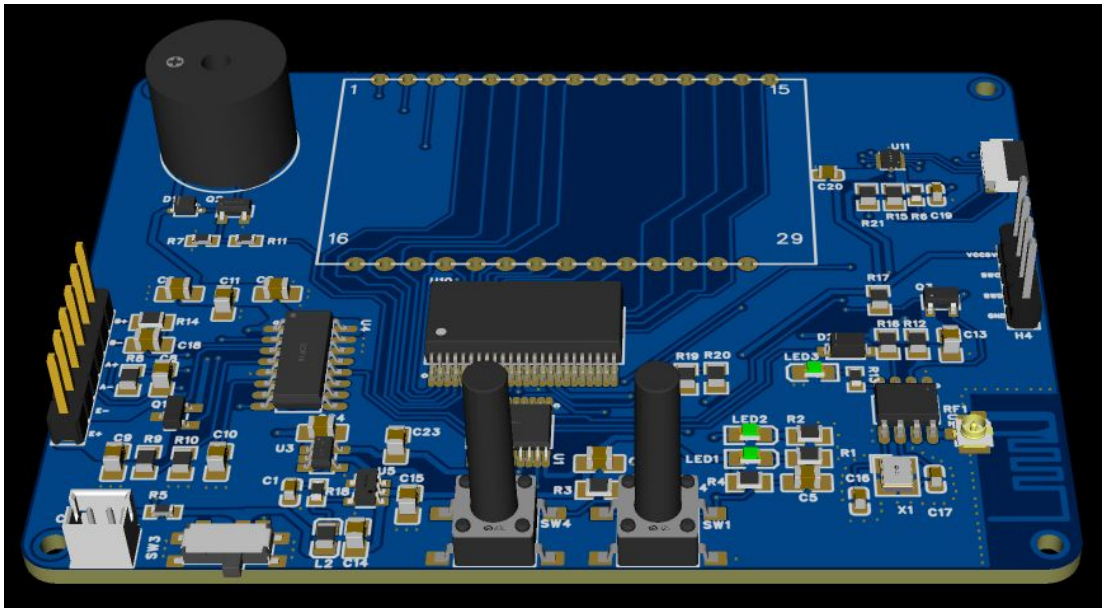
厨房电子秤
功能逻辑与代码图解

5kg厨房秤 · PY32F002A / HX711 / ECB01C

固件开发版 1.0.2 · 协议 2 · 2026年11月5日

面向程序开发与联调阅读。按启动、采样、称重、交互、通信和参数恢复的顺序，把真实源码图与它产生的实际行为配对说明。代码图保留文件名与行号，便于回到 VS Code 查阅。



> 阅读基线为当前本地固件开发版1.0.2。编译和桌面模拟已完成，实物烧录、称重精度、屏幕、电池和无线链路的验收状态在正文逐项列出。

源代码快照 scale_basic.c / SHA256 0692ed771e531d16def9

如何阅读本册

每个模块按源码图、触发条件、处理过程和结果展开。代码图由当前源文件直接生成，长行仅作视觉折行，续行不增加源代码行号。图下给出源码哈希前缀，完整快照信息见末页。

硬件输入、C 类型和默认参数 3

上电恢复、主循环和引脚初始化 6

HX711、尖峰处理、平均与判稳 10

分段标定、归零和去皮 15

错误状态、分度和重量帧 19

串口组帧、序号去重和命令 23

实体按键和蜂鸣提示 28

空秤跟踪、120秒待机和唤醒 30

双页配置、CRC和失败回退 33

看门狗、构建与验证范围 38

函数索引与来源快照 41

> 操作次序、PCB 接线、所有命令与错误处理，配套查阅第二册《PCB操作与联调指南》。本册的“预期输出”说明代码行为，实物测试状态以验证页为准。

代码如何对应到这块板

src/scale_basic.c L17-L27

```c
#define HX711_PORT GPIOA
#define HX711_DATA_PIN GPIO_PIN_0
#define HX711_CLOCK_PIN GPIO_PIN_6
#define BUZZER_PORT GPIOA
#define BUZZER_PIN GPIO_PIN_1
#define KEY_PORT GPIOB
#define KEY_PIN GPIO_PIN_3
#define LED_PORT GPIOB
#define LED_PIN GPIO_PIN_0

/* 联调默认值。稳定阈值按 ADC 计数设置，量程和实际精度需要实物确认。 */
```

源码图 src/scale_basic.c 第 17-27 行 · SHA256 0692ed771e53

实际功能

网络或器件	MCU 引脚	代码中的用途
HX711 DOUT / PD_SCK	PA0 / PA6	读取称重计数；时钟拉高还用于待机掉电
SW4 / LED2	PB3 / PB0	功能键输入 / 运行心跳
蜂鸣器驱动	PA1	有源通断或无源方波
ECB01C 串口	PA2 TX / PA3 RX	9600bps、8N1 收发
SWD	PA13 / PA14	保留给下载和调试
两块显示接口	PA4 / PB2 片选	初始化为高，显示驱动待接入

宏名称把物理引脚集中在文件开头。后续阅读 KEY_PIN 或 HX711_CLOCK_PIN 时，可回到这里定位。电池检测接 PB1，当前采样和百分比换算尚未实现。[H1]

配置变量与统一重量状态

include/weight_state.h L28-L44

```
28 typedef struct {
29     int32_t net_mg;              /* 未应用显示分度的净重。 */
30     int32_t gross_mg;            /* 包含容器的毛重。 */
31     int32_t tare_mg;             /* 皮重。 */
32     int32_t display_mg;          /* 最后取整后的显示重量。 */
33     uint32_t sequence;           /* 发送重量帧的序号。 */
34     uint32_t unit;               /* 0 为 g，1 为 kg，内部值始终以 mg 表示。 */
35     uint16_t division_mg;        /* 当前分度，100/500/1000mg。 */
36     int8_t battery_percent;      /* -1 为未知，取得真实电池换算参数后再填入。 */
37     bool stable;
38     WeightStatus status;
39     WeightMotion motion;
40 } WeightState;
41
42 extern WeightState weight_state;
43
44 #endif
```

源码图 include/weight_state.h 第 28-44 行 · SHA256 d49d46442fe9

实际功能

WeightState 像一张当前测量状态表。net_mg、gross_mg 和 tare_mg 分别是未量化净重、毛重和皮重，display_mg 才是按分度取整后的数值。蓝牙使用这张表生成重量帧，屏幕驱动接入后也读取它。

表达式	用白话理解
battery_percent = -1	尚未得到电量数据，界面可显示“未知”或 --
division_mg = 100	分度100mg，即0.1g；这项数值描述显示步长
ScaleConfig saved_config	Flash 中参数的运行副本，保存标定、单位和待机等设置
static	文件级变量只在当前 .c 文件内部直接访问
typedef / enum / bool	给类型起名 / 定义可选状态 / 表示真或假

默认数值分别控制什么

src/scale_basic.c L29-L46

```c
    AVERAGE_SAMPLES = 8,          /* 稳定时平均 8 点，降低显示抖动。 */
    FAST_SAMPLES = 3,             /* 变化时平均最近 3 点，缩短跟随延迟。 */
    HX711_STARTUP_MS = 500,       /* 给上电后的 HX711 留出建立时间。 */
    HX711_TIMEOUT_MS = 1000,      /* 连续 1 秒无有效采样即报告故障。 */
    STABLE_HOLD_MS = 500,         /* 整个窗口持续满足阈值 500ms 才算稳定。 */
    DEFAULT_STABLE_COUNTS = 64,  /* 最近 8 点的最大值与最小值之差。 */
    DEFAULT_REPORT_MS = 200,      /* 文档要求 5～10Hz，默认选择 5Hz。 */
    MIN_REPORT_MS = 100,
    MAX_REPORT_MS = 200,
    KEY_DEBOUNCE_MS = 30,
    KEY_LONG_PRESS_MS = 1500,
    BUZZER_ON_MS = 120,
    BUZZER_GAP_MS = 100,
    ALARM_INTERVAL_MS = 2000,
    UART_BAUD = 9600,
    MAX_CONFIG_VALUE = 1000000,
    MAX_GROSS_GRAMS = 5000,
    OL_GRAMS = 5250,
```

源码图 src/scale_basic.c 第 29-46 行 · SHA256 0692ed771e53

实际功能

参数	对应行为
8点 / 3点	稳定时平均8点，动态时平均最近3点
500ms / 1000ms	上电建立时间 / 连续无有效读数的故障门限
64计数、500ms	8点峰峰值不超过64且持续500ms后判稳
200ms，允许100～200ms	默认每秒5帧，可设每秒5～10帧
30ms / 1500ms	按键消抖 / 长按识别
120ms / 100ms	每声持续时间 / 声间间隔

> 64 的单位是 ADC 计数。每克对应多少计数由传感器和标定决定，需要在样机上测噪声并调整。

上电后先恢复哪些东西

src/scale_basic.c L1021-L1043

```c
int main(void)
{
    HAL_Init();
    board_init();
    scale_config_load(&saved_config);
    stable_counts = saved_config.stable_counts;
    buzzer_mode = (BuzzerMode)saved_config.buzzer_mode;
    zero_counts = saved_config.zero_counts;
    if (saved_config.point_count != 0U) {
        unsigned last = saved_config.point_count - 1U;
        reference_grams = saved_config.grams[last];
        zero_set = true;
        empty_reference_known = true;
    }
    started_ms = HAL_GetTick();
    last_sample_ms = started_ms;
    uint32_t report_ms = started_ms;
    uint32_t led_ms = started_ms;
    uint32_t power_ms = started_ms;
    idle_since_ms = started_ms;
    /* 独立看门狗由 LSI 驱动，约 4 秒未回到主循环时复位，实际时长受 LSI 误差影响。 */
    __HAL_DBGMCU_FREEZE_IWDG(); /* SWD 暂停内核时冻结看门狗，便于再次下载和调试。 */
    watchdog.Instance = IWDG;
```

源码图 src/scale_basic.c 第 1021-1043 行 · SHA256 0692ed771e53

实际功能

先初始化 HAL 和板级引脚，再读取双页 Flash 配置。有效标定存在时，恢复空秤基线与标定点，允许进入克数换算；没有有效标定时，恢复默认设置并上报 UNCAL。

皮重是本次运行的状态，上电初值为0。去皮序号缓存、消息序号、临时上报设置同样从初始值开始。已有标定只提供空秤参考，开机放着0.5g物体时仍会保留这个真实负载。



主循环怎样并行照顾各功能

src/scale_basic.c L1048-L1074

```
1048    for (;;) {
1049        uint32_t now = HAL_GetTick();
1050        if (watchdog_ready) {
1051            HAL_IWDG_Refresh(&watchdog);
1052        }
1053        sample_poll(now);
1054        key_poll(now);
1055        uart_poll(now);
1056        if ((uint32_t)(now - power_ms) >= 10U) {
1057            power_ms = now;
1058            power_poll(now);
1059            alarm_poll(now);
1060        }
1061        buzzer_poll(now);
1062        if (!sleeping && (uint32_t)(now - led_ms) >= 500U) {
1063            led_ms = now;
1064            HAL_GPIO_TogglePin(LED_PORT, LED_PIN);
1065        }
1066        if (stream_enabled && (uint32_t)(now - report_ms) >= report_interval_ms &&
    tx_next == NULL) {
1067            report_ms = now;
1068            tx_next = weight_report(now);
1069        }
1070        if (sleeping && tx_next == NULL) {
1071            __WFI();
1072        }
1073    }
1074 }
```

源码图 src/scale_basic.c 第 1048-1074 行 · SHA256 0692ed771e53

实际功能

每轮依次处理采样、按键、串口、蜂鸣器。电源与报警约每10ms检查一次，LED 每500ms翻转，重量按100～200ms的周期发布。每个任务快速返回，下一轮继续检查。

STREAM 0 只暂停周期上报。采样、按键、故障状态和心跳继续运行。待机时允许执行 WFI 等待中断，串口或按键活动会唤醒任务流程。

引脚初始化为何先写电平

src/scale_basic.c L119-L143

```c
static void board_init(void)
{
    __HAL_RCC_GPIOA_CLK_ENABLE();
    __HAL_RCC_GPIOB_CLK_ENABLE();
    __HAL_RCC_USART1_CLK_ENABLE();

    HAL_GPIO_WritePin(HX711_PORT, HX711_CLOCK_PIN | BUZZER_PIN, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4, GPIO_PIN_SET); /* TFT 片选空闲为高。 */
    HAL_GPIO_WritePin(LED_PORT, LED_PIN, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_2, GPIO_PIN_SET); /* 段码 LCD 片选空闲为高。 */

    GPIO_InitTypeDef pin = {
        .Mode = GPIO_MODE_OUTPUT_PP,
        .Pull = GPIO_NOPULL,
        .Speed = GPIO_SPEED_FREQ_LOW,
        .Pin = HX711_CLOCK_PIN | BUZZER_PIN | GPIO_PIN_4,
    };
    HAL_GPIO_Init(GPIOA, &pin);
    pin.Pin = LED_PIN | GPIO_PIN_2;
    HAL_GPIO_Init(GPIOB, &pin);

    pin.Mode = GPIO_MODE_INPUT;
    pin.Pin = HX711_DATA_PIN;
    HAL_GPIO_Init(HX711_PORT, &pin);
    pin.Pin = KEY_PIN;
```

源码图 src/scale_basic.c 第 119-143 行 · SHA256 0692ed771e53

实际功能

先把 HX711 时钟、蜂鸣器和 LED 输出置低，把两个显示片选置高，再配置为输出。这样在切换引脚方向时，各电路就有明确的初始状态。

PA0 配为输入读取 DOUT。PB3 使用上拉输入，SW4 按下时接地，因此程序读到低电平就认为按下。当前显示驱动尚未更新屏幕内容。

> 引脚对应原理图网络，使用内部8MHz时钟，PF0/PF1继续留给显示相关接口。PA13/PA14保留给SWD。[H1]

串口与毫秒计时怎样建立

src/scale_basic.c L147-L158

```c
    /* PA2 为 TX，PA3 为 RX，AF1。按 ECB01C 出厂设置使用 9600、8N1。 */
    USART1->CR1 = 0;
    USART1->CR2 = 0;
    USART1->CR3 = 0;
    USART1->BRR = (HAL_RCC_GetPCLK1Freq() + UART_BAUD / 2U) / UART_BAUD;
    USART1->CR1 = USART_CR1_TE | USART_CR1_RE | USART_CR1_UE;
    pin.Mode = GPIO_MODE_AF_PP;
    pin.Pin = GPIO_PIN_2 | GPIO_PIN_3;
    pin.Pull = GPIO_NOPULL;
    pin.Alternate = GPIO_AF1_USART1;
    HAL_GPIO_Init(GPIOA, &pin);
}
```

源码图 src/scale_basic.c 第 147-158 行 · SHA256 0692ed771e53

src/interrupts.c L4-L7

```c
void SysTick_Handler(void)
{
    HAL_IncTick();
}
```

源码图 src/interrupts.c 第 4-7 行 · SHA256 a1e50cd591bc

实际功能

USART1 的 PA2 为发送、PA3 为接收，使用 AF1 复用。代码把波特率设为9600，字长和校验配置形成8N1。它与ECB01C的串口侧必须匹配。

SysTick 每毫秒更新 HAL 的计数。now 与上一次时间相减，就能判断是否到达上报、消抖或提示音时刻。uint32_t 时间差支持计数回绕，代码避免长延时占住主循环。

环节	频率或单位
MCU 与 ECB01C 的串口	9600bps，传输速率
重量帧上报	默认5Hz，即每200ms一帧
HX711 新读数	由 RATE 硬件决定，设计方案为10SPS

HX711一帧读数怎样取得

src/scale_basic.c L168-L193

```c
static bool hx711_read(int32_t *counts)
{
    uint32_t bits = 0;
    uint32_t irq_mask = __get_PRIMASK();
    /* 只在短暂移位期间关中断，避免时钟高电平被中断拉长而进入掉电模式。 */
    __disable_irq();
    hx711_edge_delay();
    for (unsigned pulse = 0; pulse < 25; ++pulse) {
        HX711_PORT->BSRR = HX711_CLOCK_PIN;
        hx711_edge_delay();
        if (pulse < 24) {
            bits = (bits << 1) | ((HX711_PORT->IDR & HX711_DATA_PIN) != 0U);
        }
        HX711_PORT->BSRR = (uint32_t)HX711_CLOCK_PIN << 16;
        hx711_edge_delay();
    }
    bool released = (HX711_PORT->IDR & HX711_DATA_PIN) != 0U;
    __set_PRIMASK(irq_mask);

    adc_saturated = bits == 0x800000U || bits == 0x7fffffU;
    if (!released || adc_saturated) {
        return false;
    }
    *counts = (bits & 0x800000U) ? (int32_t)bits - 0x1000000 : (int32_t)bits;
    return true;
}
```

源码图 src/scale_basic.c 第 168-193 行 · SHA256 0692ed771e53

实际功能

DOUT 就绪后，前24个时钟读出有符号采样值，第25个脉冲选择下一次的A通道128倍增益。最高位为1时把24位值扩展成负的 int32_t 计数。

短暂移位期间保存中断屏蔽状态并关中断，防止时钟高电平被中断拖长。读到正/负满码，或第25个脉冲后DOUT未释放，会判定本次数据异常。实际高低脉宽仍需示波器验证。

等待采样和超时怎样处理

src/scale_basic.c L261-L288

```c
static void sample_poll(uint32_t now)
{
    if (sleeping) {
        return;
    }
    /* 启动延时只执行一次，毫秒计数回绕后继续正常采样。 */
    if (!hx711_started) {
        if ((uint32_t)(now - started_ms) < HX711_STARTUP_MS) {
            return;
        }
        hx711_started = true;
    }
    if ((uint32_t)(now - last_sample_ms) >= HX711_TIMEOUT_MS) {
        sample_fault = true;
        sample_reset();
    }
    if ((HX711_PORT->IDR & HX711_DATA_PIN) != 0U) {
        return;
    }

    int32_t counts;
    if (!hx711_read(&counts)) {
        sample_fault = true;
        sample_reset();
        return;
    }
    sample_add(counts, now);
}
```

源码图 src/scale_basic.c 第 261-288 行 · SHA256 0692ed771e53

实际功能

启动建立时间只执行一次。DOUT仍为高时，本轮立即返回；有新结果才移位读取。连续1秒没有有效结果时，清空旧窗口并置故障，避免拿旧重量做去皮或归零。

待机时暂停读取。故障恢复后，必须重新积累有效样本；满码、超时和读取错误都通过统一状态转成 ERR，正常重量字段留空。

三点中值怎样去掉尖峰

src/scale_basic.c L208-L236

```c
static void sample_add(int32_t counts, uint32_t now)
{
    last_sample_ms = now;
    sample_fault = false;
    adc_saturated = false;
    latest_raw = counts;
    raw_samples[raw_index] = counts;
    raw_index = (raw_index + 1U) % 3U;
    if (raw_count < 3U) {
        ++raw_count;
        if (raw_count < 3U) {
            return;
        }
    }
    /* 三点中值抑制单个尖峰，连续加料和持续阶跃仍会进入后续滤波。 */
    int32_t a = raw_samples[0];
    int32_t b = raw_samples[1];
    int32_t c = raw_samples[2];
    if (a > b) { int32_t swap = a; a = b; b = swap; }
    if (b > c) { b = c; }
    counts = a > b ? a : b;
    if (sample_count == AVERAGE_SAMPLES) {
        sample_sum -= samples[sample_index];
    } else {
        ++sample_count;
    }
    samples[sample_index] = counts;
    sample_sum += counts;
    sample_index = (sample_index + 1U) % AVERAGE_SAMPLES;
```

源码图 src/scale_basic.c 第 208-236 行 · SHA256 0692ed771e53

实际功能

先保留最近3个原始计数，再取中间大小的那个值送入平均窗口。例如100、101、7000的中值是101，单个7000尖峰不会直接拉动显示。

持续加载会连续改变多个采样，仍能进入后续计算。此处没有把低于2g的重量硬设为0，也没有按重量大小丢弃小信号。latest_raw 同时保留最新原始值，供诊断和近零检查。

最近8点怎样判断稳定

src/scale_basic.c L237-L258

```
237
238     int32_t minimum = samples[0];
239     int32_t maximum = samples[0];
240     for (unsigned i = 1; i < sample_count; ++i) {
241         if (samples[i] < minimum) {
242             minimum = samples[i];
243         }
244         if (samples[i] > maximum) {
245             maximum = samples[i];
246         }
247     }
248     if (sample_count < AVERAGE_SAMPLES || (uint32_t)(maximum - minimum) > stable_counts
    ) {
249         settling = false;
250         stable = false;
251         return;
252     }
253     if (!settling) {
254         settling = true;
255         stable_since_ms = now;
256     }
257     stable = (uint32_t)(now - stable_since_ms) >= STABLE_HOLD_MS;
258 }
```

源码图 src/scale_basic.c 第 237-258 行 · SHA256 0692ed771e53

实际功能

环形窗口保存最近8点。每次收入一个值，就查窗口的最小值和最大值。窗口不足8点，或峰峰值超过stable_counts，就清除稳定候选。

窗口满足条件后开始计时，连续500ms保持合格才置 stable。窗口1000～1040的峰峰值为40，低于默认64计数；这仍需持续满足时间条件。加载或振动超过阈值会立即取消稳定标志。

动态跟随和稳定平滑怎样切换

src/scale_basic.c L304-L315

```c
static int32_t filtered_counts(void)
{
    if (stable) {
        return average_counts();
    }
    int32_t total = 0;
    for (unsigned i = 0; i < FAST_SAMPLES; ++i) {
        unsigned index = (sample_index + AVERAGE_SAMPLES - 1U - i) % AVERAGE_SAMPLES;
        total += samples[index];
    }
    return total / FAST_SAMPLES;
}
```

源码图 src/scale_basic.c 第 304-315 行 · SHA256 0692ed771e53

src/scale_basic.c L291-L295

```c
static bool samples_ready(uint32_t now)
{
    return sample_count >= FAST_SAMPLES && !sample_fault &&
           (uint32_t)(now - last_sample_ms) < HX711_TIMEOUT_MS;
}
```

源码图 src/scale_basic.c 第 291-295 行 · SHA256 0692ed771e53

实际功能

动态时取最近3点平均，缩短连续加料的拖尾；stable 成立后取完整窗口平均，减小抖动。标定、归零和去皮需要稳定窗口，统一用完整平均值建立基线。

samples_ready 还检查采样是否新鲜、是否有故障。刚启动或恢复时要先积累至少3点有效滤波样本。窗口和时序决定的是算法响应，整机稳定时间还受机械结构、传感器与环境影响。

ADC计数怎样变成克数

src/scale_basic.c L318-L334

```c
static int64_t gross_milligrams(int32_t counts)
{
    int direction = saved_config.spans[0] < 0 ? -1 : 1;
    int64_t x = ((int64_t)counts - zero_counts) * direction;
    int64_t start_x = 0;
    int64_t start_mg = 0;
    for (unsigned i = 0; i < saved_config.point_count; ++i) {
        int64_t end_x = (int64_t)saved_config.spans[i] * direction;
        int64_t end_mg = (int64_t)saved_config.grams[i] * 1000;
        if (x <= end_x || i + 1U == saved_config.point_count) {
            return start_mg + (x - start_x) * (end_mg - start_mg) / (end_x - start_x);
        }
        start_x = end_x;
        start_mg = end_mg;
    }
    return 0;
}
```

源码图 src/scale_basic.c 第 318-334 行 · SHA256 0692ed771e53

实际功能

空秤计数为原点，标定点保存砝码质量和对应计数差。程序找到相邻两个点，在这一小段内按比例插值；低于零点和超过末点时，沿对应一段的斜率外推。

例如500g和2500g两个标定点之间，计数刚好在中间时换算为1500g。计数方向允许为负，适配加载后ADC数值下降的接法。中间计算用 int64_t，输出先保留毫克，后续才应用显示分度。

> 曲线正确需要真实砝码和正确安装的传感器。固件支持三点修正，实际误差仍需按0.5g至5000g的验收点测量。

CAL怎样录入标定点

src/scale_basic.c L680-L705

```c
static const char *calibrate(uint32_t now)
{
    uint32_t grams;
    if (!parse_value(rx_line + 4, &grams) || grams == 0U || grams > MAX_GROSS_GRAMS ||
  !zero_set) {
        return "ERR,CAL_NEEDS_ZERO_1_TO_5000G\r\n";
    }
    /* 标定依据已确认的砝码质量，旧曲线的偏差不能阻止满量程标定。 */
    const char *error = measurement_error(now, false);
    if (error != NULL) {
        return error;
    }
    ScaleConfig candidate = saved_config;
    unsigned index = 0;
    while (index < candidate.point_count && candidate.grams[index] < grams) {
        ++index;
    }
    if (index == 3U) {
        return "ERR,CAL_MAX_3_POINTS\r\n";
    }
    candidate.zero_counts = zero_counts;
    candidate.spans[index] = average_counts() - zero_counts;
    candidate.grams[index] = grams;
    candidate.point_count = index + 1U;
    if (!scale_config_valid(&candidate)) {
        return "ERR,CAL_SPAN_OR_ORDER\r\n";
    }
```

源码图 src/scale_basic.c 第 680-705 行 · SHA256 0692ed771e53

实际功能

命令接受1 ~ 5000的整数克数，要求已经建立空秤零点、最近数据新鲜且稳定。最多保存3点，录入同一或更小质量时，后续标定点作废，需要重新录入。

初步曲线可能把真实5000g算得略大。标定命令依据操作者已确认的砝码质量进行修正，因此不会被旧曲线的超量程结果阻止；计数方向、单调性和存储校验仍必须通过。

```
CAL 500
CAL 2500
CAL 5000
OK,CAL,SAVED
```

ZERO怎样保护真实小负载

src/scale_basic.c L377-L397

```c
static const char *set_zero(uint32_t now)
{
    const char *error = measurement_error(now, true);
    if (error != NULL) {
        return error;
    }
    /* 已标定时只能在原空秤基线附近归零，防止抹掉 0.5g 等真实小负载。 */
    if (reference_grams != 0U) {
        int64_t gross = gross_milligrams(average_counts());
        if (gross < -100 || gross > 100) {
            return "ERR,NOT_EMPTY\r\n";
        }
    }
    zero_counts = average_counts();
    zero_set = true;
    empty_reference_known = true;
    empty_candidate = false;
    tare_counts = 0;
    weight_update(now);
    return "OK,ZERO\r\n";
}
```

源码图 src/scale_basic.c 第 377-397 行 · SHA256 0692ed771e53

实际功能

归零前检查新鲜、稳定和毛重上限。已有标定时，还要求原空秤基线附近的毛重位于±0.1g内。0.5g物体会得到 NOT_EMPTY，避免被归零操作吞掉。

成功后更新运行零点，保留标定曲线并清除皮重。第一次尚未标定时，ZERO是操作者对空秤的确认，随后需要用真实砝码完成CAL。普通称重过程中，负重量和小重量继续按换算结果发布。

TARE与ZERO分别改变什么

src/scale_basic.c L400-L415

```c
static const char *set_tare(uint32_t now)
{
    if (reference_grams == 0U) {
        return "ERR,NEED_CAL\r\n";
    }
    const char *error = measurement_error(now, true);
    if (error != NULL) {
        return error;
    }
    int32_t counts = average_counts();
    tare_counts = counts - zero_counts;
    empty_candidate = false;
    filter_seed(counts, now);
    weight_update(now);
    return "OK,TARE\r\n";
}
```

源码图 src/scale_basic.c 第 400-415 行 · SHA256 0692ed771e53

src/scale_basic.c L337-L343

```c
static void filter_seed(int32_t counts, uint32_t now)
{
    sample_reset();
    for (unsigned i = 0; i < FAST_SAMPLES + 2U; ++i) {
        sample_add(counts, now);
    }
}
```

源码图 src/scale_basic.c 第 337-343 行 · SHA256 0692ed771e53

实际功能

TARE先要求已有标定，再检查新鲜、稳定和5kg毛重上限。成功时记录当前容器偏移，并用这次稳定平均值重建滤波窗口，立即得到零净重。

后续先分别换算毛重与皮重，再相减。这样在分段曲线下也能正确去皮。取消去皮把偏移设为0；移走已经去皮的容器，允许产生负净重。失败时旧皮重保留。

什么时候发布WAIT或ERR

src/scale_basic.c L530-L553

```
530 static void weight_update(uint32_t now)
531 {
532     weight_state.unit = saved_config.unit;
533     weight_state.stable = false;
534     if (sleeping) {
535         weight_state.status = WEIGHT_SLEEP;
536         weight_state.motion = MOTION_SLEEP;
537         return;
538     }
539     if (sample_fault || adc_saturated || (uint32_t)(now - last_sample_ms) >=
    HX711_TIMEOUT_MS) {
540         weight_state.status = WEIGHT_SENSOR_ERROR;
541         weight_state.motion = MOTION_FAULT;
542         return;
543     }
544     if (!samples_ready(now)) {
545         weight_state.status = WEIGHT_WAIT;
546         weight_state.motion = MOTION_CANDIDATE;
547         return;
548     }
549     if (reference_grams == 0U) {
550         weight_state.status = WEIGHT_UNCALIBRATED;
551         weight_state.motion = MOTION_CANDIDATE;
552         return;
553     }
```

源码图 src/scale_basic.c 第 530-553 行 · SHA256 0692ed771e53

实际功能

条件	统一状态
软件待机	SLEEP
HX711超时、满码或读取失败	ERR
有效样本还不足	WAIT
没有有效标定点	UNCAL

这些分支先把稳定标志清除，并停止发布正常重量数字。状态对象里可能仍保留上一次数值，接收端应先检查 status；蓝牙报文会主动把无效重量字段留空。

5kg上限与OL怎样区分

src/scale_basic.c L554-L575

```
554     int64_t gross = gross_milligrams(filtered_counts());
555     int64_t tare = gross_milligrams(zero_counts + tare_counts);
556     int64_t net = gross - tare;
557     if (gross >= (int64_t)OL_GRAMS * 1000) {
558         weight_state.status = WEIGHT_OVERLOAD;
559         weight_state.motion = MOTION_FAULT;
560         return;
561     }
562     if (gross > (int64_t)MAX_GROSS_GRAMS * 1000 || gross < -(int64_t)MAX_GROSS_GRAMS *
    1000 ||
563         net < INT32_MIN || net > INT32_MAX || tare < INT32_MIN || tare > INT32_MAX) {
564         weight_state.status = WEIGHT_RANGE;
565         weight_state.motion = MOTION_FAULT;
566         return;
567     }
568     weight_state.motion = net < weight_state.net_mg ? MOTION_UNLOADING : MOTION_LOADING
    ;
569     weight_state.gross_mg = (int32_t)gross;
570     weight_state.tare_mg = (int32_t)tare;
571     weight_state.net_mg = (int32_t)net;
572     weight_state.division_mg = select_division((int32_t)net);
573     int64_t magnitude = net < 0 ? -net : net;
574     int64_t rounded = ((magnitude + weight_state.division_mg / 2) / weight_state.
    division_mg) * weight_state.division_mg;
575     weight_state.display_mg = net < 0 ? -(int32_t)rounded : (int32_t)rounded;
```

源码图 src/scale_basic.c 第 554-575 行 · SHA256 0692ed771e53

实际功能

毛重	状态和输出
≤5000g且其他检查通过	OK，可发布正常重量
>5000g且<5250g	RANGE，正常重量字段留空
≥5250g	OL，正常重量字段留空

毛重包含容器。例如3000g容器已去皮，再加入2001g物体，总负载5001g，仍会进入RANGE。归零、去皮保留5kg限制；在已确认砝码质量下执行标定，可以修正旧曲线偏差。

分度和四舍五入放在哪一步

src/scale_basic.c L516-L527

```c
static uint16_t select_division(int32_t net_mg)
{
    int64_t magnitude = net_mg < 0 ? -(int64_t)net_mg : net_mg;
    uint16_t division = weight_state.division_mg;
    if (magnitude > 1001000 || (division == 1000 && magnitude >= 999000)) {
        return 1000;
    }
    if (magnitude > 200500 || (division >= 500 && magnitude >= 199500)) {
        return 500;
    }
    return 100;
}
```

源码图 src/scale_basic.c 第 516-527 行 · SHA256 0692ed771e53

src/scale_basic.c L574-L579

```c
    int64_t rounded = ((magnitude + weight_state.division_mg / 2) / weight_state.division_mg) * weight_state.division_mg;
    weight_state.display_mg = net < 0 ? -(int32_t)rounded : (int32_t)rounded;
    weight_state.stable = stable;
    weight_state.status = WEIGHT_OK;
    if (stable) {
        weight_state.motion = MOTION_STABLE;
```

源码图 src/scale_basic.c 第 574-579 行 · SHA256 0692ed771e53

实际功能

先完成标定和去皮，再按绝对净重选择分度，最后对正负数对称取整。基本分度为0.1g、0.5g、1g。边界有小滞回，避免阈值附近来回切换。

当前变化	分度选择
0.1g分度向上越过200.5g	切换到0.5g
0.5g分度向下低于199.5g	回到0.1g
向上越过1001g / 向下低于999g	切换到1g / 回到0.5g

重量状态怎样变成一帧数据

src/scale_basic.c L595-L614

```c
static const char *weight_report(uint32_t now)
{
    static const char *const statuses[] = {"WAIT", "UNCAL", "OK", "RANGE", "OL", "ERR",
  "SLEEP"};
    weight_update(now);
    ++weight_state.sequence;
    if (weight_state.status != WEIGHT_OK) {
        snprintf(tx_line, sizeof(tx_line), "W,%lu,,0,,%d,%s\r\n",
                 (unsigned long)weight_state.sequence, (int)weight_state.
  battery_percent,
                 statuses[weight_state.status]);
        return tx_line;
    }
    char net[20];
    char tare[20];
    format_grams(net, sizeof(net), weight_state.display_mg);
    format_grams(tare, sizeof(tare), weight_state.tare_mg);
    snprintf(tx_line, sizeof(tx_line), "W,%lu,%s,%u,%s,%d,OK\r\n",
             (unsigned long)weight_state.sequence, net, (unsigned)weight_state.stable,
             tare, (int)weight_state.battery_percent);
    return tx_line;
}
```

源码图 src/scale_basic.c 第 595-614 行 · SHA256 0692ed771e53

实际功能

每次生成重量帧，序号加1。正常帧包含净重、稳定、皮重、电量和OK。电量当前为-1，表示尚未取得百分比。g/kg是显示设置，串口协议的重量字段统一使用克。

```
W,1024,136.4,1,120.0,-1,OK

W,1025,,0,,-1,OL
```

示例第一帧表示净重136.4g、皮重120g，毛重为256.4g。第二帧表示超载，两个重量字段留空。READ也会生成这种帧，周期刷新由STREAM和RATE控制。

UART怎样收到一条完整命令

src/scale_basic.c L890-L916

```c
static void uart_poll(uint32_t now)
{
    uint32_t status = USART1->SR;
    const uint32_t errors = USART_SR_ORE | USART_SR_NE | USART_SR_FE | USART_SR_PE;
    if ((status & (USART_SR_RXNE | errors)) != 0U) {
        idle_since_ms = now;
        if (sleeping) { wake_scale(now); }
        uint8_t byte = (uint8_t)USART1->DR; /* 读 SR 后读 DR，清除接收错误标志。 */
        if (!rx_ready) {
            if ((status & errors) != 0U) {
                rx_invalid = true;
            } else if (byte == '\r' || byte == '\n') {
                if (rx_length != 0U || rx_invalid) {
                    rx_line[rx_length] = '\0';
                    rx_ready = true;
                }
            } else if (byte < 32U || byte > 126U || rx_length == sizeof(rx_line) - 1U)
 {
                rx_invalid = true;
            } else if (!rx_invalid) {
                rx_line[rx_length++] = (char)byte;
            }
        }
    }
    if (tx_next == NULL && rx_ready) {
        tx_next = handle_command(now);
        /* 带序号去皮在去重分支内提示。其他命令成功一声、失败三声，查询结果保持安静。 */
        if (strncmp(rx_line, "TARE,", 5) != 0) {
```

源码图 src/scale_basic.c 第 890-916 行 · SHA256 0692ed771e53

实际功能

逐字节接收ASCII，遇到CR或LF才结束一行。CRLF中的空换行会被忽略。收到非打印字符、超长行或串口错误后，整行一直作废到行尾，避免截断文本被当成有效命令。

接收缓冲最多47个字符，另留一个字符串结束符。收到任何串口数据都会更新无操作时间，待机中还会先唤醒。设备只保留一条等待执行的命令，平板应在完整应答后发送下一条。

命令执行和发送怎样衔接

src/scale_basic.c L917-L933

```
917             if (strncmp(tx_next, "ERR,", 4) == 0) {
918                 buzzer_start(3, now);
919             } else if (strncmp(tx_next, "OK,", 3) == 0) {
920                 buzzer_start(1, now);
921             }
922         }
923         rx_length = 0;
924         rx_ready = false;
925         rx_invalid = false;
926     }
927     if (tx_next != NULL && (status & USART_SR_TXE) != 0U) {
928         USART1->DR = (uint8_t)*tx_next++;
929         if (*tx_next == '\0') {
930             tx_next = NULL;
931         }
932     }
933 }
```

源码图 src/scale_basic.c 第 917-933 行 · SHA256 0692ed771e53

实际功能

上一段发送完成后，才执行已接收的命令并取得应答指针。TXE允许发送时，每轮只写一个字节。共用发送缓冲在这段时间保持该帧内容，完整发完再接下一条。

普通命令返回OK时一声，返回ERR时三声，正常查询保持安静。带序号TARE由专门去重分支提示，重复应答不重复去皮或响铃。静音模式下仍会发送相同的成功或错误报文。

去皮序号怎样阻止重复执行

src/scale_basic.c L651-L677

```c
static const char *sequenced_tare(uint32_t now)
{
    uint32_t sequence;
    if (!parse_sequence(rx_line + 5, &sequence)) {
        buzzer_start(3, now);
        return "ERR,TARE,?,BAD_SEQ\r\n";
    }
    if (tare_sequence_seen && sequence == last_tare_sequence) {
        return last_tare_reply;
    }
    if (tare_sequence_seen && (uint32_t)(sequence - last_tare_sequence) >= 0x80000000U) {
        snprintf(tx_line, sizeof(tx_line), "ERR,TARE,%lu,STALE_SEQ\r\n", (unsigned long)sequence);
        return tx_line;
    }
    const char *result = set_tare(now);
    if (result[0] == 'O') {
        snprintf(last_tare_reply, sizeof(last_tare_reply), "ACK,TARE,%lu\r\n", (unsigned long)sequence);
        buzzer_start(1, now);
    } else {
        snprintf(last_tare_reply, sizeof(last_tare_reply), "ERR,TARE,%lu,%.*s\r\n",
                 (unsigned long)sequence, (int)strlen(result) - 6, result + 4);
        buzzer_start(3, now);
    }
    last_tare_sequence = sequence;
    tare_sequence_seen = true;
    return last_tare_reply;
}
```

源码图 src/scale_basic.c 第 651-677 行 · SHA256 0692ed771e53

实际功能

重发最近一次序号，直接返回保存的原应答；更早序号返回STALE_SEQ。新序号才会实际调用去皮。成功和失败的应答都会缓存，因此失败后重新尝试应使用新序号。

序号范围为0～4294967295，允许从最大值回绕到0。去重缓存属于本次运行的RAM状态，掉电后重新建立交互。简单TARE保留给手动联调，平板应使用带序号形式。

命令分派与查询入口

src/scale_basic.c L716-L738

```c
static const char *handle_command(uint32_t now)
{
    if (rx_invalid) {
        return "ERR,COMMAND\r\n";
    }
    if (strcmp(rx_line, "HELP") == 0) {
        return "CMDS,ZERO,CAL <g>,CALRESET,TARE[,seq],UNTARE,READ,RAW,STATUS,INFO,"
               "RATE <100..200>,STREAM <0|1>,BUZZER <OFF|ACTIVE|PASSIVE>,"
               "UNIT <g|kg>,AUTO_OFF <0..86400>,STABLE <counts>,DEVICE <id>,CALDATE <YYYYMMDD>,WAKE\r\n";
    }
    if (strcmp(rx_line, "READ") == 0) {
        return weight_report(now);
    }
    if (strcmp(rx_line, "RAW") == 0) {
        if (!samples_ready(now)) {
            return "ERR,WAIT_SAMPLES\r\n";
        }
        snprintf(tx_line, sizeof(tx_line), "RAW,%ld\r\n", (long)latest_raw);
        return tx_line;
    }
    if (strcmp(rx_line, "STATUS") == 0) {
        return status_report(now);
    }
```

源码图 src/scale_basic.c 第 716-738 行 · SHA256 0692ed771e53

实际功能

类别	入口与结果
查询	HELP命令列表；STATUS运行状态；INFO版本与编号
测量	READ重量帧；RAW最新有效原始计数
测量操作	ZERO、CAL、CALRESET、TARE、UNTARE
运行设置	RATE、STREAM、WAKE与固定LIMIT兼容入口
持久设置	STABLE、BUZZER、UNIT、AUTO_OFF、DEVICE、CALDATE

命令名称按大小写精确匹配。数字参数要求非负整数，单位、符号或多余空格会被拒绝。完整参数范围和应答字串见操作指南的命令页。

设置为何先保存再生效

src/scale_basic.c L781-L798

```
781     if (strncmp(rx_line, "STABLE ", 7) == 0) {
782         uint32_t value;
783         if (!parse_value(rx_line + 7, &value) || value == 0U) {
784             return "ERR,STABLE_COUNTS\r\n";
785         }
786         ScaleConfig candidate = saved_config;
787         candidate.stable_counts = value;
788         if (!save_settings(&candidate)) {
789             return "ERR,FLASH\r\n";
790         }
791         sample_reset();
792         return "OK,STABLE\r\n";
793     }
794     if (strncmp(rx_line, "RATE ", 5) == 0) {
795         uint32_t value;
796         if (!parse_value(rx_line + 5, &value) || value < MIN_REPORT_MS || value >
    MAX_REPORT_MS) {
797             return "ERR,RATE_100_TO_200_MS\r\n";
798         }
```

源码图 src/scale_basic.c 第 781-798 行 · SHA256 0692ed771e53

src/scale_basic.c L346-L355

```
346 static bool save_settings(const ScaleConfig *candidate)
347 {
348     if (!scale_config_save(candidate)) {
349         return false;
350     }
351     saved_config = *candidate;
352     stable_counts = candidate->stable_counts;
353     buzzer_mode = (BuzzerMode)candidate->buzzer_mode;
354     return true;
355 }
```

源码图 src/scale_basic.c 第 346-355 行 · SHA256 0692ed771e53

实际功能

先在candidate副本里修改并检查参数，再写备用Flash页。新记录通过读回验证后，才替换运行配置；失败返回ERR,FLASH，旧设置继续生效。

STABLE修改后清空采样，重新积累窗口。UNIT、蜂鸣器、AUTO_OFF、设备编号、日期和标定参数会保存。RATE、STREAM、皮重、去皮缓存及运行中的零点微调保存在RAM。

SW4怎样区分短按和长按

src/scale_basic.c L456-L484

```c
static void key_poll(uint32_t now)
{
    static bool last_pressed;
    static bool debounced_pressed;
    static bool long_handled;
    static uint32_t changed_ms;
    static uint32_t pressed_ms;
    const char *result = NULL;
    bool pressed = (KEY_PORT->IDR & KEY_PIN) == 0U;
    if (pressed) { idle_since_ms = now; }

    if (pressed != last_pressed) {
        last_pressed = pressed;
        changed_ms = now;
    }
    if (pressed != debounced_pressed && (uint32_t)(now - changed_ms) >= KEY_DEBOUNCE_MS
  ) {
        debounced_pressed = pressed;
        if (pressed) {
            pressed_ms = now;
            long_handled = false;
            if (sleeping) {
                wake_scale(now);
                long_handled = true; /* 唤醒用的这次按键在释放前只做唤醒。 */
            }
        } else if (!long_handled) {
            result = set_tare(now);
        }
    }
    if (debounced_pressed && pressed && !long_handled &&
```

源码图 src/scale_basic.c 第 456-484 行 · SHA256 0692ed771e53

按下和释放都要连续保持30ms才接受。短按在释放后去皮；按住至1.5秒执行空秤归零，每次长按只处理一次，释放后不再去皮。

待机时，第一次按下只唤醒。pressed_ms、long_handled记住本次按压进度。按键操作成功一声，失败三声，实际错误仍由稳定、新鲜度、空秤与总负载条件决定。

蜂鸣器如何分段响而不阻塞

src/scale_basic.c L431-L453

```c
static void buzzer_poll(uint32_t now)
{
    if (buzzer_remaining == 0U) {
        return;
    }
    uint32_t duration = buzzer_on ? BUZZER_ON_MS : BUZZER_GAP_MS;
    if ((uint32_t)(now - buzzer_phase_ms) >= duration) {
        buzzer_phase_ms = now;
        buzzer_tone_ms = now;
        if (buzzer_on) {
            --buzzer_remaining;
            buzzer_on = false;
            HAL_GPIO_WritePin(BUZZER_PORT, BUZZER_PIN, GPIO_PIN_RESET);
        } else {
            buzzer_on = true;
            HAL_GPIO_WritePin(BUZZER_PORT, BUZZER_PIN, GPIO_PIN_SET);
        }
    } else if (buzzer_on && buzzer_mode == BUZZER_PASSIVE &&
               (uint32_t)(now - buzzer_tone_ms) >= 1U) {
        buzzer_tone_ms = now;
        HAL_GPIO_TogglePin(BUZZER_PORT, BUZZER_PIN);
    }
}
```

源码图 src/scale_basic.c 第 431-453 行 · SHA256 0692ed771e53

buzzer_start记录剩余声数并拉高PA1。buzzer_poll按毫秒切换120ms发声和100ms间隔，期间继续采样、收发和检测按键。

模式或事件	代码行为
OFF	默认静音，保留协议结果和错误状态
ACTIVE	高电平持续通断，配合有源蜂鸣器
PASSIVE	约每1ms翻转，形成约500Hz联调音调
持续传感器故障或超载	每2秒启动三声提示

哪些条件允许慢速零点跟踪

src/scale_basic.c L969-L996

```c
static void power_poll(uint32_t now)
{
    if (sleeping) {
        return;
    }
    weight_update(now);
    bool empty = empty_reference_known && tare_counts == 0 && weight_state.status == WEIGHT_OK &&
                 weight_state.stable && weight_state.net_mg >= -100 && weight_state.net_mg <= 100;
    if (empty) {
        int64_t raw_mg = gross_milligrams(latest_raw);
        empty = raw_mg >= -100 && raw_mg <= 100;
    }
    if (!empty) {
        empty_candidate = false;
        idle_since_ms = now;
        return;
    }
    if (!empty_candidate) {
        empty_candidate = true;
        empty_since_ms = now;
        zero_track_ms = now;
    }
    if ((uint32_t)(now - empty_since_ms) < EMPTY_CONFIRM_MS) {
        return;
    }
    weight_state.motion = MOTION_EMPTY;
    if ((uint32_t)(now - zero_track_ms) >= 1000U) {
        /* 每秒只修正偏差的 1/64，跟踪量留在 RAM，避免频繁擦写 Flash。 */
```

源码图 src/scale_basic.c 第 969-996 行 · SHA256 0692ed771e53

需要已有空秤参考、皮重为0、重量有效且稳定，滤波净重与最新原始换算值都在±0.1g内。满足后进入候选，连续至少3秒才允许慢速跟踪。

任一条件失效就取消候选并重置无操作时间。0.5g、1g、2g负载会阻止跟踪与自动待机。平板连续向串口发送查询，也会持续重置无操作计时。

120秒后具体关闭哪些部分

src/scale_basic.c L997-L1004

```
997         zero_counts += (average_counts() - zero_counts) / 64;
998         zero_track_ms = now;
999     }
1000    if (saved_config.auto_off_seconds != 0U &&
1001        (uint32_t)(now - idle_since_ms) >= saved_config.auto_off_seconds * 1000U) {
1002        enter_standby();
1003    }
1004 }
```

源码图 src/scale_basic.c 第 997-1004 行 · SHA256 0692ed771e53

src/scale_basic.c L936-L947

```
936 static void enter_standby(void)
937 {
938     sleeping = true;
939     sample_reset();
940     HAL_GPIO_WritePin(HX711_PORT, HX711_CLOCK_PIN, GPIO_PIN_SET);
941     HAL_GPIO_WritePin(LED_PORT, LED_PIN, GPIO_PIN_RESET);
942     HAL_GPIO_WritePin(BUZZER_PORT, BUZZER_PIN, GPIO_PIN_RESET);
943     buzzer_remaining = 0;
944     weight_state.status = WEIGHT_SLEEP;
945     weight_state.motion = MOTION_SLEEP;
946     weight_state.stable = false;
947 }
```

源码图 src/scale_basic.c 第 936-947 行 · SHA256 0692ed771e53

近零确认后，每秒只修正ADC偏差的1/64，修正留在RAM。自动待机时间默认120秒，AUTO_OFF 0可关闭这项定时功能。

进入待机后，PA6时钟持续拉高使HX711掉电，LED2与蜂鸣输出关闭，重量状态切到SLEEP。MCU继续保留按键、串口和毫秒中断，电源与蓝牙支路仍按硬件连接工作。

唤醒后为何先出现WAIT

src/scale_basic.c L950-L966

```c
static void wake_scale(uint32_t now)
{
    if (!sleeping) {
        return;
    }
    sleeping = false;
    HAL_GPIO_WritePin(HX711_PORT, HX711_CLOCK_PIN, GPIO_PIN_RESET);
    sample_reset();
    sample_fault = false;
    adc_saturated = false;
    hx711_started = false;
    started_ms = now;
    last_sample_ms = now;
    idle_since_ms = now;
    empty_candidate = false;
    weight_update(now);
}
```

源码图 src/scale_basic.c 第 950-966 行 · SHA256 0692ed771e53

唤醒将HX711时钟拉低，清空旧窗口，重置启动建立时间和错误标志，等待新的有效采样。状态先回到WAIT，再按是否已标定进入OK或UNCAL。

SW4或任何串口数据都可唤醒，WAKE命令会得到OK,WAKE。自动待机保持RAM，复位则会重新读取Flash。两者对皮重、序号缓存和临时上报设置的影响不同，详见操作指南。

哪些配置会掉电保存

include/scale_config.h L8-L23

```h
typedef struct {
    int32_t zero_counts;
    int32_t spans[3];
    uint32_t grams[3];
    uint32_t point_count;
    uint32_t stable_counts;
    uint32_t auto_off_seconds;
    uint32_t buzzer_mode;
    uint32_t unit;              /* 0 为 g, 1 为 kg。蓝牙重量帧统一以 g 传输。 */
    uint32_t calibration_date;  /* YYYYMMDD, 0 表示尚未录入。 */
    char device_id[16];
} ScaleConfig;

void scale_config_defaults(ScaleConfig *config);
bool scale_config_valid(const ScaleConfig *config);
bool scale_config_load(ScaleConfig *config);
```

源码图 include/scale_config.h 第 8-23 行 · SHA256 e2606eaeb28f

src/scale_config.c L43-L49

```c
void scale_config_defaults(ScaleConfig *config)
{
    memset(config, 0, sizeof(*config));
    config->stable_counts = 64;
    config->auto_off_seconds = 120;
    strcpy(config->device_id, "UNSET");
}
```

源码图 src/scale_config.c 第 43-49 行 · SHA256 e9c5a1be8557

配置保存空秤标定计数、最多3个质量点、判稳阈值、待机秒数、蜂鸣模式、显示单位、标定日期和设备编号。新设备默认阈值64、待机120秒、静音、g单位、编号UNSET、日期0。

运行皮重、RATE、STREAM、去皮应答缓存和慢速零点微调均属于RAM状态。手动ZERO的运行零点也在下次重启时恢复为最后一次标定保存的零点。

CRC如何识别未完成的记录

src/scale_config.c L29-L40

```c
static uint32_t config_crc(const void *data, size_t length)
{
    const uint8_t *bytes = data;
    uint32_t crc = UINT32_MAX;
    for (size_t i = 0; i < length; ++i) {
        crc ^= bytes[i];
        for (unsigned bit = 0; bit < 8; ++bit) {
            crc = (crc >> 1) ^ (0xEDB88320U & (0U - (crc & 1U)));
        }
    }
    return ~crc;
}
```

源码图 src/scale_config.c 第 29-40 行 · SHA256 e9c5a1be8557

src/scale_config.c L77-L82

```c
static bool record_valid(const ConfigRecord *record)
{
    return record->magic == CONFIG_MAGIC && record->version == CONFIG_VERSION &&
           record->crc == config_crc(record, offsetof(ConfigRecord, crc)) &&
           scale_config_valid(&record->config);
}
```

源码图 src/scale_config.c 第 77-82 行 · SHA256 e9c5a1be8557

记录包含标识、格式版本、递增代数、固件版本、配置和CRC32。把前面数据计算出的CRC与保存值比较，只有一致且各字段有效时，记录才参与恢复。

字段检查还要求标定质量递增、计数方向一致且段间跨度足够，避免损坏数据形成零除或无效曲线。CRC是完整性检查，真正掉电与Flash电气行为仍需在样机上测试。

两份记录如何选新与回退

src/scale_config.c L85-L96

```
85 static int current_slot(void)
86 {
87     const ConfigRecord *first = (const ConfigRecord *)SCALE_CONFIG_BASE;
88     const ConfigRecord *second = (const ConfigRecord *)(SCALE_CONFIG_BASE +
   CONFIG_PAGE_SIZE);
89     bool first_valid = record_valid(first);
90     bool second_valid = record_valid(second);
91     if (!first_valid) {
92         return second_valid ? 1 : -1;
93     }
94     uint32_t difference = second->generation - first->generation;
95     return second_valid && difference != 0U && difference < 0x80000000U ? 1 : 0;
96 }
```

源码图 src/scale_config.c 第 85-96 行 · SHA256 e9c5a1be8557

src/scale_config.c L99-L109

```
99  bool scale_config_load(ScaleConfig *config)
100 {
101     int slot = current_slot();
102     if (slot < 0) {
103         scale_config_defaults(config);
104         return false;
105     }
106     const ConfigRecord *record = (const ConfigRecord *)(SCALE_CONFIG_BASE + (unsigned)
    slot * CONFIG_PAGE_SIZE);
107     *config = record->config;
108     return true;
109 }
```

源码图 src/scale_config.c 第 99-109 行 · SHA256 e9c5a1be8557

仅一页有效就用该页；两页都有效时选代数较新的一页。代数差按无符号数判断，支持回绕。两页都无效时恢复默认值并等待标定。

上电不会把上一份无效数据直接当成重量系数。桌面模拟覆盖了半页写入和新页CRC损坏后的回退，实物断电恢复仍列为待验收项。

保存前怎样保留旧记录

src/scale_config.c L112-L139

```
112 bool scale_config_save(const ScaleConfig *config)
113 {
114     if (!scale_config_valid(config)) {
115         return false;
116     }
117     int active = current_slot();
118     uint32_t generation = 1;
119     if (active >= 0) {
120         const ConfigRecord *old = (const ConfigRecord *)(SCALE_CONFIG_BASE + (unsigned)
      active * CONFIG_PAGE_SIZE);
121         if (memcmp(&old->config, config, sizeof(*config)) == 0) {
122             return true;
123         }
124         generation = old->generation + 1U;
125     }
126     unsigned target = active == 0 ? 1U : 0U;
127     uintptr_t address = SCALE_CONFIG_BASE + target * CONFIG_PAGE_SIZE;
128     uint32_t page[CONFIG_PAGE_SIZE / sizeof(uint32_t)];
129     memset(page, 0xFF, sizeof(page));
130     ConfigRecord record = {
131         .magic = CONFIG_MAGIC,
132         .version = CONFIG_VERSION,
133         .generation = generation,
134         .firmware_version = FIRMWARE_VERSION,
135         .config = *config,
136     };
137     record.crc = config_crc(&record, offsetof(ConfigRecord, crc));
138     memcpy(page, &record, sizeof(record));
139
```

源码图 src/scale_config.c 第 112-139 行 · SHA256 e9c5a1be8557

配置先做范围和单调性检查。与当前有效配置相同时跳过写入；有变化时递增代数，选择另一页作目标，把完整记录放进128字节的RAM页缓冲。

目标页始终是备用页，当前有效页在本次保存中保留。程序区只占前16KB，最后4KB扇区专用于参数，避免普通固件扇区擦写覆盖配置。

写入成功的判断放在哪一步

src/scale_config.c L140-L159

```
    if (HAL_FLASH_Unlock() != HAL_OK) {
        return false;
    }
    FLASH_EraseInitTypeDef erase = {
        .TypeErase = FLASH_TYPEERASE_PAGEERASE,
        .PageAddress = address,
        .NbPages = 1,
    };
    uint32_t page_error;
    HAL_StatusTypeDef status = HAL_FLASHEx_Erase(&erase, &page_error);
    if (status == HAL_OK) {
        status = HAL_FLASH_Program(FLASH_TYPEPROGRAM_PAGE, address, page);
    }
    /* 固定版本 SDK 的编程错误出口可能保留软件锁，串行调用结束后恢复该锁。 */
    pFlash.Lock = HAL_UNLOCKED;
    HAL_FLASH_Lock();
    /* 以读回的完整记录为提交结果，兼容数据已写完但驱动随后报告错误的情况。 */
    return record_valid((const ConfigRecord *)address) &&
           memcmp((const void *)address, &record, sizeof(record)) == 0;
}
```

源码图 src/scale_config.c 第 140-159 行 · SHA256 e9c5a1be8557

解锁后按128字节页擦除，再写入新页并锁回Flash。最后读回整个记录，比较头部、数据和CRC，确认与本次候选记录完全一致，才应用到运行状态。

驱动报错但完整记录已经落盘时，以读回验证作为提交结果。数据不完整时继续使用旧页。Flash编程例程由链接脚本复制到RAM执行，上传参数使用扇区擦除。

主循环卡住时怎样恢复

src/scale_basic.c L1041-L1055

```
1041    /* 独立看门狗由 LSI 驱动，约 4 秒未回到主循环时复位，实际时长受 LSI 误差影响。 */
1042    __HAL_DBGMCU_FREEZE_IWDG(); /* SWD 暂停内核时冻结看门狗，便于再次下载和调试。 */
1043    watchdog.Instance = IWDG;
1044    watchdog.Init.Prescaler = IWDG_PRESCALER_256;
1045    watchdog.Init.Reload = 511;
1046    watchdog_ready = HAL_IWDG_Init(&watchdog) == HAL_OK;
1047
1048    for (;;) {
1049        uint32_t now = HAL_GetTick();
1050        if (watchdog_ready) {
1051            HAL_IWDG_Refresh(&watchdog);
1052        }
1053        sample_poll(now);
1054        key_poll(now);
1055        uart_poll(now);
```

源码图 src/scale_basic.c 第 1041-1055 行 · SHA256 0692ed771e53

src/interrupts.c L10-L15

```
10  void HardFault_Handler(void)
11  {
12      while (1) {
13          /* 保留异常现场，等待调试器。 */
14      }
15  }
```

源码图 src/interrupts.c 第 10-15 行 · SHA256 a1e50cd591bc

IWDG使用独立LSI时钟，配置256分频与511重装值，约4秒没有喂狗会复位。主循环每轮刷新，HardFault停留或任务长期卡住时会失去刷新机会。实际时间受LSI误差影响。

SWD暂停内核时冻结看门狗，便于下载和调试。软件待机仍运行主循环并等待中断，因此会正常喂狗。看门狗复位后的参数恢复由前面的双页配置流程负责。

点灯目标和称重目标怎样选择

platformio.ini L23-L29

```
[env:scale_py32f002a]
; 仅保留点灯入口 main.c，两个环境分别链接自己的 main 函数。
build_src_filter = +<*> -<scale_basic.c>

[env:scale_basic]
; 使用 scale_basic.c 作为入口，包含采样、称重、校准、按键和提示音。
build_src_filter = +<*> -<main.c>
```

源码图 platformio.ini 第 23-29 行 · SHA256 bb56c9228844

tools/pio_upload.py L13-L19

```
env.Replace(
    UPLOADCMD='"$PYTHONEXE" "{}" load --target py32f002ax5 '
    '--pack "{}" --erase sector --frequency 1000000 -O connect_mode=halt '
    '-O reset_type=sysresetreq -O auto_unlock=false "$SOURCE"'.format(
        uploader, pack
    )
)
```

源码图 tools/pio_upload.py 第 13-19 行 · SHA256 0e7ede557bb6

区域或目标	用途
scale_py32f002a	默认点灯目标，只运行PB0翻转入口
scale_basic	称重、协议、按键、蜂鸣、存储与待机入口
0x08000000～0x08003FFF	16KB程序区
0x08004000起	4KB配置扇区，双页位于0x4000/0x4080偏移

上传使用pyOCD、官方DFP与1MHz SWD。setup_pack.py下载并核对官方包的SHA256；该下载校验与设备配置CRC用途不同。

哪些结论已经检查过

```
scale_py32f002a SUCCESS Flash 1660B RAM 1008B
scale_basic SUCCESS Flash 14632B RAM 2104B
Simulated settling at 10SPS: 1400ms
```

检查类别	本轮结果
实际构建	两个PIO环境编译并链接成功，原SDK/newlib链接警告仍存在
质量与曲线	三点标定、非线性去皮、满量程旧曲线误判修正
状态与交互	0.5g保留、负重量、边界分度、总负载、实体键、蜂鸣
通信	CRLF、超长行、接收错误、去皮去重与uint32回绕
参数可靠性	有效记录内每字节位置截断、CRC损坏、擦写失败与读回提交
电源与时间	0.5g阻止吸零、120秒待机、UART唤醒和毫秒回绕

> 1.4秒来自无机械振动的算法模拟。整机±0.2/±1/±3g、10分钟漂移、四角偏载、BLE 8小时和整机24小时运行需要真实样机与砝码验收。

屏幕驱动、段映射、电池ADC与低电提醒、RK3566端应用仍待接入。电量字段当前为-1，表示未知。

函数索引一

以下页码定位功能解释，源码图中可直接查看真实行号。辅助函数与其调用场景一起说明。

函数	职责	页码
board_init	GPIO和串口初始配置	8
hx711_edge_delay	8MHz下提供短边沿间隔	10
hx711_read	24位读取、符号和满码检查	10
sample_reset	清空原始和滤波窗口	11
sample_add	中值、滑窗、峰峰值与稳定计时	12
sample_poll	就绪读取和超时恢复	11
samples_ready	有效样本与新鲜度检查	14
average_counts	完整窗口平均	14
filtered_counts	动态3点、稳定8点	14
gross_milligrams	分段曲线换算	15
filter_seed	去皮后重建窗口	18
save_settings	保存成功后应用配置	27
overloaded	5kg毛重限制	20
measurement_error	就绪、稳定和可选量程检查	17
set_zero / set_tare	空秤归零 / 容器去皮	17
buzzer_start / buzzer_poll	提示音启动和非阻塞音型	29
key_poll	消抖、短长按和待机唤醒	28
parse_value	0 ~ 1000000严格整数解析	26
select_division	分度滞回	21
weight_update	统一状态与最终取整	19

函数索引二

函数	职责	页码
format_grams	把mg格式化为一位小数g	22
weight_report	生成W重量或故障帧	22
status_report	查询运行标志和参数	26
parse_sequence	完整uint32序号解析	25
sequenced_tare	去皮应答缓存和旧序号拒绝	25
calibrate	标定点录入与保存	16
handle_command	全部命令分派	26
uart_poll	逐字节收发和音型反馈	23
enter_standby / wake_scale	待机进入 / 唤醒重建采样	31
power_poll / alarm_poll	近零跟踪、待机和故障三声	30
main	恢复配置与轮询调度	6
config_crc / record_valid	记录完整性与字段检查	34
scale_config_defaults	默认配置	33
scale_config_valid	字段范围与曲线单调性	34
current_slot	选择最新有效记录	35
scale_config_load	恢复配置或默认值	35
scale_config_save	双页保存与读回验证	36
SysTick / HardFault	毫秒基准 / 保留故障现场	38
点灯main / 系统文件封装	最小运行验证 / SDK时钟实现	39

依据与代码快照

[D1] 用户确认的《厨房电子秤产品需求规格书》5kg V1.0，2026年10月30日。
[D2] 用户确认的《厨房电子秤项目设计说明书》5kg V1.0，2026年10月30日。
[H1] 用户提供的电子秤原理图，Board1 / P1 / V1.0。
[H2] 嘉立创项目3D视图与用户实物图，用于器件定位。

[C1] 本地 py32-scale-pio 工程。图下SHA前缀定位文件内容，完整SHA256列于下表。
[T1] 本次PIO实际编译日志。
[T2] work/scale-host-check/check.c，模拟HAL、UART、ADC和Flash的回归检查。

文件	SHA256
src/scale_basic.c	0692ed771e531d16def909994164a03833ed73a6c06d3f836b70d3d412ee7650
src/scale_config.c	e9c5a1be85572e7337b86624dd93095080173d8105bd0a2db4ebdb3cb36d95a6
include/weight_state.h	d49d46442fe9fed2513042d8dce78a2f31efc661317435ad1051d5e25ed1e6e5
platformio.ini	bb56c9228844d823362fe826a74e672a292ab2290e840c4d3604a3cddf612bb3
tools/pio_upload.py	0e7ede557bb6a212794e68c0bf2c88d3a71f4470167cd6613f7b13e46fea5ff2

> 本册记录当前开发快照。修改代码后，行号、分度、协议或默认参数可能变化，应重新核对并生成手册。工程位置为 C:/Users/29268/Documents/Codex/2026-10-30/new-chat/outputs/py32-scale-pio。